

AI Use Disclosure: For Questions 1, 2, and 3, I completed the heuristic implementations and initial analysis manually. I used an AI assistant (Gemini) as a tutor and proofreader to double-check my logic (such as confirming the mathematical admissibility of the heuristics), to concisely format my data tables, and to convert my original writing into more formal versions, though all core ideas and logic are entirely my own. I verified all algorithmic steps and code functionality by running test scripts to ensure UCS and A* optimal costs matched perfectly. For Question 4, I used the AI as explicitly required by the assignment, documenting its generated heuristic, proving its admissibility, and testing its node reduction directly in my code.

Nathan Tien
Section 3

Question 1

Heuristic Implementations:

- Steps:
 - Formula/Pseudocode:

```
counter = 0
for i in range(len(n))
    if n[i] != goal[i]
        counter += 1
return counter
```
 - Unavoidable remaining cost: This heuristic lower-bounds the absolute minimum number of independent transitions required to reach the goal. Because every mismatched position must be corrected, the count of mismatches represents the bare minimum number of actions remaining.
 - Admissibility: The game rules dictate that a legal action changes exactly one letter. It is physically impossible to correct a word with k mismatched letters in fewer than k steps. The actual cost may be higher if the dictionary graph forces intermediate detours, ensuring this simple mismatch count never overestimates the true remaining optimal cost.
- Scrabble:
 - Formula/Pseudocode:

```
counter = 0
for i in range(len(n))
    if n[i] != goal[i])
        counter += scrabble_cost(goal[i])
return counter
```
 - Unavoidable remaining cost: This heuristic lower-bounds the cumulative point value of the specific target letters needed to complete the goal word.

To transform the current state into the goal, the missing letters must eventually be swapped into the string.

- Admissibility: This calculates a theoretically flawless path where only the necessary goal letters are inserted with zero wasted intermediate words. Since any required intermediate dictionary words will introduce additional letters (and therefore extra Scrabble point costs) before the final word is reached, the sum of just the missing target letters acts as a strict, impossible-to-overestimate minimum toll.
- Frequency:
 - Formula/Pseudocode:

```
counter = 0
for i in range(len(n))
    if n[i] != goal[i]
        counter += 1
return counter
```
 - Unavoidable remaining cost: This heuristic lower-bounds the cumulative base cost of taking the minimum required steps. The problem defines the cost of entering a word as $1 + \text{rarity}(w)$. Since rarity is explicitly stated to be nonnegative (0 or greater), the absolute lowest possible cost of a single valid transition is exactly 1.
 - Admissibility: By using the standard mismatch count, the heuristic assumes that the path to the goal requires the absolute minimum possible number of steps, and that every single intermediate word in that path magically has a rarity penalty of exactly 0. Because any real dictionary path will likely require more steps and incur actual rarity penalties, the true remaining cost will always be greater than or equal to this highly optimistic base count.

Question 2

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY → ASK	Steps	5	773	5	16	97.93%
WHY → ASK	Scrabble	9	765	9	13	98.30%
WHY → ASK	Frequency	38.8554	808	38.8554	755	6.56%
BEAR → DUCK	Steps	4	691	4	6	99.13%
BEAR → DUCK	Scrabble	11	2981	11	6	99.80%
BEAR → DUCK	Frequency	38.2402	1084	38.2402	720	33.58%

2.1

Yes, UCS and A* returned the exact same optimal costs. This is the expected behavior and confirms that the implemented heuristics are mathematically admissible. Because the heuristics never overestimate the true remaining cost, A* is guaranteed to find the true shortest path, perfectly matching the exhaustive search of UCS.

2.2

With a reduction of 98.3%, the highest reduction for WHY → ASK was with using A* search and Scrabble as the cost measurement.

With a reduction of 99.80%, the highest reduction for BEAR → DUCK was with using A* search and Scrabble as the cost measurement.

2.3

With a reduction of 6.56%, the least informative heuristic for WHY → ASK was with A* search using Frequency as the cost measurement.

With a reduction of 33.58%, the least informative heuristic for BEAR → DUCK was with A* search using Frequency as the cost measurement.

2.4

While A* uses a heuristic to give direction to its search compared to the blind exploration of UCS, the efficiency of that search depends heavily on the heuristic's definition. The Scrabble heuristic was highly informative because its definition incorporates the exact point values of the missing target letters, creating a lower bound that closely mirrors the actual true cost of the path. This highly accurate estimate allowed A* to aggressively prune bad pathways.

Conversely, the Frequency heuristic was the least informative because its definition, a simple mismatch count assuming every transition costs exactly 1, grossly underestimates the actual cost. The true frequency paths rack up massive rarity penalties (costs of 38+), but the heuristic only guesses a cost of 3 or 4. Because this estimate is so far below reality, it provides very little directional guidance, forcing A* to explore almost as many dead-end nodes as UCS.

Question 3

- Steps:
 - Yes or No: Yes.
 - Justification: The cost of any single legal transition is strictly 1, regardless of which letter is changing or what word you are entering. Because every step costs exactly 1, any valid sequence of words from W1 to W2 can be traversed in the exact reverse from W2 to W1 with the exact same number of steps, yielding the exact same total cost.
- Scrabble:
 - Yes or No: No.
 - Justification: The Scrabble cost is direction-dependent because you are charged based on the inserted letter (the destination state), not the removed letter. Therefore, reversing the action changes the letter being scored, which changes the edge weight.
 - Counterexample: Consider a one-step path between cat and hat.
cat → hat: You insert the letter 'h'. The Scrabble cost for 'h' is 4.
hat → cat: You insert the letter 'c'. The Scrabble cost for 'c' is 3.
The action is reversible, but the costs are asymmetric.
- Frequency:
 - Yes or No: No.

- Justification: The cost formula is $1 + \text{rarity}(w)$, where w is the destination word. Moving from a common word to a rare word forces you to pay the high rarity penalty of the rare word. Moving in reverse, from the rare word back to the common word, means you only pay the small rarity penalty of the common word. The toll is always collected at the destination, making the costs asymmetric.
- Counterexample: Consider a theoretical one-step transition between a common word like *the* and a rare word like *thy*.
 $\text{the} \rightarrow \text{thy}$: The destination is *thy*, so the cost is $1 + 10.579178 = 11.579178$.
 $\text{thy} \rightarrow \text{the}$: The destination is *the*, so the cost is $1 + 0.016226212 = 1.016226212$.
 The path is fully reversible, but the directional costs are vastly different.

Question 4

4.1

Model: Gemini

Prompt: "In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal."

Proposed heuristic: The LLM proposed adding the specific rarity value of the goal word to the baseline mismatch count. It reasoned that since the target word must unconditionally be entered as the final step of any successful path, the search will always be forced to pay that specific destination rarity penalty.

4.2

Precise formula or pseudocode:

Plaintext

None

if $n == \text{goal}$:

 return 0

else:

 return $h_0(n) + \text{rarity}(\text{goal})$

4.3

This proposed heuristic is strictly admissible. Every valid transition incurs a cost of $1 + \text{rarity}(w_{\text{next}})$. A state n that is $h_0(n)$ mismatches away from the goal requires a bare minimum of $h_0(n)$ transitions. The sum of the base cost of 1 for these required transitions is exactly $h_0(n)$. The final transition must always land on the goal state, incurring an additional known penalty of $\text{rarity}(\text{goal})$. Assuming all intermediate states have the absolute minimum possible rarity of 0, the theoretical minimum remaining cost is exactly $h_0(n) + \text{rarity}(\text{goal})$. Because the heuristic calculates this exact mathematical floor, it cannot overestimate the true optimal remaining cost.

4.4

The proposal is valid and does not require repair. It successfully meets the criteria of being "more informative" because it is at least as large as h_0 for every state, and strictly larger for any problem where $\text{rarity}(\text{goal}) > 0$.

4.5

The new rarity-aware heuristic successfully reduced the search space compared to the baseline mismatch count $h_0(n)$.

For why $\rightarrow$ ask, A* expanded 634 nodes using the new heuristic, down from 755 nodes with the baseline $h_0(n)$.

For bear $\rightarrow$ duck, A* expanded 302 nodes using the new heuristic, a massive drop from 720 nodes with the baseline $h_0(n)$.

Because the proposed heuristic successfully incorporates the destination's mandatory rarity penalty, it raises the mathematical lower bound without overestimating the true cost. This makes the heuristic strictly more informative, allowing A* to prune significantly more unpromising paths early in the search.